Vademecum D2WM-CDA - Guide Complet pour l'Examen Oral

Ce document regroupe **toutes les connaissances essentielles** pour préparer votre examen oral de Concepteur Développeur d'Applications. Chaque section contient des questions types avec des réponses structurées, des exemples concrets et des points d'attention.

📋 Table des matières

1. Questions générales sur le développement web
2. Sécurité web
3. Bases de données et SQL
4. Architecture et conception
5. Tests et qualité
6. Méthodologies de développement
7. Performance et optimisation
8. APIs et services web
9. DevOps et déploiement
10. Technologies spécifiques
11. Git et contrôle de version
12. Design Patterns
13. WebSockets et temps réel
14. PWA et applications mobiles
15. OAuth et authentification moderne

Questions générales sur le développement web {#questions-generales}

1. Expliquez le fonctionnement d'une application web moderne

Réponse structurée :

Une application web moderne suit une architecture client-serveur avec plusieurs couches :

Architecture complète :

- **Frontend** : Interface utilisateur (HTML, CSS, JavaScript/TypeScript) exécutée dans le navigateur
- **Backend** : Logique métier et accès aux données (serveur d'application)
- **Base de données** : Stockage persistant des données (SQL ou NoSQL)
- **API** : Interface de communication entre frontend et backend (REST, GraphQL, ou gRPC)
- **Cache** : Couche d'optimisation (Redis, Memcached)
- **CDN** : Distribution de contenu statique

Flux typique :

```
Client → Requête HTTP → Load Balancer → Serveur → Cache/BDD → Traitement →
Réponse → Client
```

Technologies modernes :

- **Frontend** : Angular 20, React, Vue.js avec TypeScript
- **Backend** : Node.js 24, Spring Boot, Symfony 7, Django
- **Communication** : WebSockets pour le temps réel, REST pour CRUD classique

2. Quelles sont les différences entre HTTP et HTTPS ?

Réponse :

Aspect	HTTP	HTTPS
Sécurité	Données en clair, non sécurisé	Données chiffrées avec SSL/TLS
Port	80	443
Certificat	Non requis	Certificat SSL/TLS obligatoire
Performance	Légèrement plus rapide	Overhead minimal avec HTTP/2
SEO	Pénalisé par Google	Favorisé dans le classement
Authentification	Aucune	Vérification de l'identité du serveur

Points importants :

- HTTPS protège contre l'interception (man-in-the-middle)
- Obligatoire pour les données sensibles (RGPD)
- Améliore la confiance des utilisateurs
- Requis pour les PWA et Service Workers

3. Qu'est-ce que le responsive design et pourquoi est-ce important ?

Réponse :

Le responsive design permet à un site web de s'adapter automatiquement à différentes tailles d'écran et orientations.

Techniques principales :

- **Media queries CSS** : Conditions basées sur la taille d'écran

  ```css
  @media (max-width: 768px) {
    .container { width: 100%; }
  }
  ```

- **Grilles flexibles** : CSS Grid, Flexbox

- **Images adaptatives** : `srcset`, `picture`, formats modernes (WebP, AVIF)
- **Unités relatives** : rem, em, %, vw, vh au lieu de px fixes
- **Mobile-first** : Conception pour mobile d'abord, puis desktop

Importance stratégique :

- **Expérience utilisateur** : 60%+ du trafic web est mobile
- **Coûts** : Une seule version au lieu de mobile + desktop
- **SEO** : Google utilise l'indexation mobile-first depuis 2019
- **Conversion** : Sites responsive = meilleurs taux de conversion

Breakpoints standards :

- Mobile : < 576px
- Tablette : 576px - 992px
- Desktop : > 992px

4. Comment gérez-vous l'accessibilité dans vos applications web ?

Réponse :

L'accessibilité suit les directives **WCAG 2.1** (Web Content Accessibility Guidelines) avec 4 principes POUR :

P - **Perceptible** :

- Alternatives textuelles (attributs `alt` pour images)
- Sous-titres pour vidéos et transcriptions audio
- Contrastes de couleurs suffisants (4.5:1 pour texte normal, 3:1 pour texte large)

O - **Opérable** :

- Navigation au clavier complète (Tab, Shift+Tab, Enter, Espace)
- Pas de piège au clavier
- Temps suffisant pour lire et interagir

U - **Understandable (Compréhensible)** :

- Langue de la page déclarée (`<html lang="fr">`)
- Messages d'erreur clairs
- Navigation cohérente

R - **Robuste** :

- HTML sémantique (`<nav>`, `<main>`, `<article>`, `<aside>`)
- Attributs ARIA pour les composants complexes

```
<button aria-label="Fermer le menu" aria-expanded="true">
```

Outils de test :

- **Lighthouse** (Chrome DevTools) : Score d'accessibilité

- **axe DevTools** : Extension navigateur
- **WAVE** : Évaluation visuelle
- **Lecteurs d'écran** : NVDA (Windows), VoiceOver (Mac)

5. Expliquez le processus de rendu d'une page web dans le navigateur

Réponse :

Le **Critical Rendering Path** comprend 6 étapes :

1. Parsing HTML → DOM

- Le navigateur analyse le HTML
- Construit le DOM (Document Object Model)

2. Parsing CSS → CSSOM

- Analyse des feuilles de style
- Construction du CSSOM (CSS Object Model)

3. Render Tree

- Combinaison DOM + CSSOM
- Calcul des styles appliqués à chaque élément visible

4. Layout (Reflow)

- Calcul des positions et dimensions exactes
- Processus coûteux en performance

5. Paint

- Remplissage des pixels
- Création des couches (layers)

6. Composite

- Assemblage des couches
- Affichage final à l'écran

Optimisations critiques :

- **Critical CSS** : CSS inline pour le contenu above-the-fold
- **Defer/Async JavaScript** : Ne pas bloquer le rendu

```
<script src="app.js" defer></script>
```

- **Minification** : Réduction de la taille CSS/JS (30-50%)
- **Lazy loading** : Images et iframes hors écran
- **Preload** : Ressources critiques

```
<link rel="preload" href="font.woff2" as="font">
```

Métriques de performance :

- **FCP** (First Contentful Paint) : < 1.8s
- **LCP** (Largest Contentful Paint) : < 2.5s
- **CLS** (Cumulative Layout Shift) : < 0.1
- **FID** (First Input Delay) : < 100ms

Sécurité web {#securite}

1. Quelles sont les principales vulnérabilités web (OWASP Top 10 2021) ?

Réponse :

Top 10 OWASP actualisé :

1. **Broken Access Control** 🔴
   - Accès non autorisé à des ressources
   - Exemple : Modifier l'URL `/user/123` en `/user/124` pour voir d'autres profils
2. **Cryptographic Failures** (ex-Sensitive Data Exposure)
   - Données sensibles non chiffrées
   - Algorithmes de chiffrement obsolètes (MD5, SHA1)
3. **Injection** (SQL, NoSQL, OS, LDAP)
   - Insertion de code malveillant via les entrées
   - Toujours n°1 en dangerosité pratique
4. **Insecure Design**
   - Failles dès la conception
   - Manque d'analyse de risques
5. **Security Misconfiguration**
   - Paramètres par défaut non modifiés
   - Messages d'erreur verbeux en production
   - Ports/services inutiles ouverts
6. **Vulnerable and Outdated Components**
   - Utilisation de dépendances avec failles connues
   - Importance des mises à jour régulières
7. **Identification and Authentication Failures**
   - Session hijacking, brute force

    - Absence de MFA (Multi-Factor Authentication)

8. **Software and Data Integrity Failures**
    - CI/CD non sécurisés
    - Désérialisation non fiable
9. **Security Logging and Monitoring Failures**
    - Logs insuffisants ou inexploitables
    - Détection d'intrusion absente
10. **Server-Side Request Forgery (SSRF)**
    - Le serveur effectue des requêtes non autorisées
    - Accès à des ressources internes

2. Comment prévenir les injections SQL ?

Réponse :

Techniques de protection (du plus au moins efficace) :

1. Requêtes préparées (Prepared Statements) ✅ MEILLEUR

```
// PHP PDO
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = ? AND password =
?");
$stmt->execute([$email, $hashedPassword]);
```

```
// Java JDBC
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users WHERE
id = ?");
stmt.setInt(1, userId);
ResultSet rs = stmt.executeQuery();
```

Pourquoi c'est efficace :

- Séparation stricte code SQL / données
- Le SGBD compile la requête avant d'insérer les données
- Impossible d'injecter du code SQL dans les paramètres

2. ORM (Object-Relational Mapping)

```
// TypeORM (Node.js)
const user = await userRepository.findOne({
  where: { email: email }
});
```

```
// Doctrine (Symfony)
$user = $entityManager->getRepository(User::class)->findOneBy(['email' =>
$email]);
```

3. Validation et sanitisation des entrées

- **Whitelisting** (liste blanche) plutôt que blacklisting
- Validation du type de données (entier, email, etc.)
- Limitation de la longueur

4. Principe du moindre privilège

- Compte BDD avec droits minimaux
- Pas de `DROP`, `ALTER` pour l'application
- Séparation lecture/écriture si possible

5. Procédures stockées (avec précautions)

```
CREATE PROCEDURE GetUser(IN userId INT)
BEGIN
  SELECT * FROM users WHERE id = userId;
END;
```

× À NE JAMAIS FAIRE :

```
// VULNÉRABLE
$query = "SELECT * FROM users WHERE name = '$name'";
// Si $name = "' OR '1'='1' --" → contournement authentification
```

Exemple d'attaque :

```
-- Login : admin' --
-- Password : n'importe quoi
SELECT * FROM users WHERE login='admin' --' AND password='...'
-- Le -- met le reste en commentaire !
```

3. Expliquez les attaques XSS et leurs préventions

Réponse :

3 types d'XSS (Cross-Site Scripting) :

1. Stored XSS (Persistant) 🔴 PLUS DANGEREUX

- Script malveillant stocké en base de données

- Exemple : Commentaire contenant `<script>alert(document.cookie)</script>`
- Exécuté pour tous les visiteurs de la page

2. Reflected XSS (Non-persistant)

- Script dans l'URL, renvoyé par le serveur
- Exemple : `https://site.com/search?q=<script>steal()</script>`
- Nécessite que la victime clique sur un lien malveillant

3. DOM-based XSS

- Manipulation du DOM côté client
- JavaScript lit l'URL et modifie le DOM sans validation
- Exemple : `document.getElementById('div').innerHTML = location.hash;`

Préventions essentielles :

1. Échappement des données (Output Encoding) ✅ PRIORITÉ

```
// PHP
echo htmlspecialchars($userInput, ENT_QUOTES, 'UTF-8');

// JavaScript
const escapeHtml = (text) => {
  const map = { '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'":
'&#039;' };
  return text.replace(/[&<>"']/g, m => map[m]);
};
```

2. Content Security Policy (CSP) 🛡️

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://trusted-cdn.com;
  object-src 'none';
  base-uri 'self';
  require-trusted-types-for 'script';
```

Bloque :

- Scripts inline (`<script>`)
- `eval()`, `setTimeout(string)`
- Scripts de domaines non autorisés

3. HttpOnly et Secure cookies

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
```

- **HttpOnly** : Cookie inaccessible via `document.cookie`
- **Secure** : Envoi uniquement en HTTPS
- **SameSite** : Protection CSRF

4. Validation des entrées

```
// Whitelist de caractères autorisés
const isValidUsername = (name) => /^[a-zA-Z0-9_-]{3,20}$/.test(name);
```

5. Sanitisation HTML

```
// DOMPurify (bibliothèque recommandée)
import DOMPurify from 'dompurify';
const clean = DOMPurify.sanitize(dirtyHtml);
```

6. Templates auto-escapés

- **Twig (Symfony)** : Échappe par défaut
- **Angular** : Sanitization automatique
- **React** : Protection intégrée avec JSX

4. Qu'est-ce que le CSRF et comment s'en protéger ?

Réponse :

CSRF (Cross-Site Request Forgery) :

Attaque forçant un utilisateur authentifié à exécuter une action non désirée sur un site où il est connecté.

Scénario d'attaque complet :

1. **Victime connectée** sur `banque.com` (cookie de session actif)
2. **Visite site malveillant** `evil.com`
3. **Site malveillant envoie requête** vers `banque.com/transfer?to=attaquant&amount=1000`
4. **Navigateur joint automatiquement** le cookie de session
5. **Action exécutée** à l'insu de la victime

Code d'attaque exemple :

```
<!-- Sur evil.com -->
<img src="https://banque.com/transfer?to=attaquant&amount=1000" />
<!-- Ou avec form auto-submit -->
<form action="https://banque.com/transfer" method="POST">
  <input name="to" value="attaquant" />
  <input name="amount" value="1000" />
</form>
<script>document.forms[0].submit();</script>
```

Protections efficaces :

1. Tokens CSRF (Synchronizer Token Pattern) ✅ MEILLEUR

```
<!-- Formulaire avec token -->
<form method="POST" action="/transfer">
  <input type="hidden" name="csrf_token" value="abc123xyz789" />
  <input name="amount" />
  <button>Transférer</button>
</form>
```

```
// Vérification côté serveur
if ($_POST['csrf_token'] !== $_SESSION['csrf_token']) {
  die('CSRF token invalide');
}
```

Caractéristiques du token :

- Unique par session ou par requête
- Imprévisible (généré aléatoirement)
- Vérifié côté serveur
- ⚠ **PAS** basé sur "action + id" (prévisible)

2. SameSite Cookies 🍪

```
Set-Cookie: sessionId=abc; SameSite=Strict
```

Valeur	Comportement
Strict	Cookie jamais envoyé en cross-site (même lien cliqué)
Lax	Cookie envoyé uniquement GET top-level (défaut moderne)
None	Cookie toujours envoyé (requiert Secure)

3. Double Submit Cookie

```
// Cookie: csrf_token=abc123
// Header: X-CSRF-Token: abc123
// Le serveur vérifie que les deux correspondent
```

4. Vérification du Referer/Origin

```
$allowed = 'https://monsite.com';
if (!str_starts_with($_SERVER['HTTP_REFERER'], $allowed)) {
  die('Referer invalide');
}
```

⚠ **Limites** : Referer peut être absent ou modifié

5. Re-authentification pour actions sensibles

- Demander le mot de passe pour transferts d'argent
- Confirmation par email/SMS

Framework automatique :

- **Symfony** : Protection CSRF intégrée dans les formulaires
- **Angular** : `HttpClientXsrfModule` automatique
- **Spring Security** : `CsrfFilter` activé par défaut

5. Comment sécuriser le stockage des mots de passe ?

Réponse :

Principes fondamentaux :

× NE JAMAIS :

- Stocker en clair
- Utiliser MD5 ou SHA1 (cassables en quelques secondes)
- Chiffrer (réversible = risque si clé compromise)

✅ TOUJOURS : Hacher avec sel et facteur de coût

Algorithmes recommandés (2024) :

1. Argon2 🥇 MEILLEUR (Winner PHC 2015)

```
// PHP
$hash = password_hash($password, PASSWORD_ARGON2ID);
$valid = password_verify($password, $hash);
```

Avantages :

- Résistant aux attaques GPU/ASIC
- Paramétrable (mémoire + temps)
- Recommandé par OWASP

2. bcrypt 🥈 EXCELLENT (Standard actuel)

```
// Node.js
const bcrypt = require('bcrypt');
const saltRounds = 12;
const hash = await bcrypt.hash(password, saltRounds);
const match = await bcrypt.compare(password, hash);
```

```
// Java (Spring Security)
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(12);
String hash = encoder.encode(password);
boolean matches = encoder.matches(password, hash);
```

Facteur de coût (work factor) :

- **10** : Rapide, minimum acceptable
- **12** : Bon équilibre (recommandé 2024)
- **14+** : Très sécurisé, plus lent

⏱️ **Temps de hachage cible : 250-500ms** (Ajuster le facteur de coût selon la puissance serveur)

3. PBKDF2 🥉 Acceptable

```
# Python
from hashlib import pbkdf2_hmac
hash = pbkdf2_hmac('sha256', password.encode(), salt, 100000)
```

Comparaison :

Algorithme	Résistance GPU	Résistance ASIC	Recommandation
Argon2id	☆☆☆☆☆	☆☆☆☆☆	Privilégier
bcrypt	☆☆☆☆	☆☆☆	Excellent
PBKDF2	☆☆	☆	Acceptable
MD5/SHA1	×	×	Obsolète

Salt (Sel) :

```
Hash = Algorithm(Password + Salt)
```

Caractéristiques du sel :

- **Unique** par utilisateur (pas de sel global)
- **Aléatoire** (cryptographiquement sûr)
- **Généré automatiquement** par bcrypt/Argon2

- **Stocké avec le hash** (pas secret)

Exemple de hash complet (bcrypt) :

```
$2y$12$abcdefghijklmnopqrstuv.WXYZ0123456789ABCDEFGHIJKLMNOPQ
 |  |  |                    |
 |  |  Salt (22 chars)    Hash (31 chars)
 |  Coût (2^12 = 4096 itérations)
 Algorithme (2y = bcrypt)
```

Politique de mots de passe (ANSSI 2024) :

- **Longueur minimale** : 12 caractères (8 si MFA activée)
- **Complexité** : Mixte majuscules, minuscules, chiffres, symboles
- **Pas de mots du dictionnaire**
- **MFA recommandée** pour comptes sensibles
- **Rotation** : Uniquement si compromission suspectée

6. Que signifie le principe de 'défense en profondeur' en sécurité ?

Réponse :

La **défense en profondeur** (defense in depth) est une stratégie multicouche où plusieurs mécanismes de sécurité se complètent.

Principe : Si une couche est compromise, les autres protègent encore le système.

Les 7 couches de sécurité :

1. Couche Humaine 👥

- Formation sécurité des développeurs
- Sensibilisation des utilisateurs
- Processus de revue de code

2. Couche Périmétrique 🛡️

- Pare-feu (Firewall)
- Protection DDoS
- WAF (Web Application Firewall)

3. Couche Réseau 🌐

- Segmentation réseau (VLANs)
- IDS/IPS (Intrusion Detection/Prevention)
- VPN pour accès distants

4. Couche Système 🖥️

- Serveurs durcis (hardening)
- Mises à jour automatiques

- Antivirus/EDR

5. Couche Application 🔐

- Validation des entrées
- Authentification forte (MFA)
- Gestion de sessions sécurisée
- Protection OWASP Top 10

6. Couche Données 💾

- Chiffrement au repos (AES-256)
- Chiffrement en transit (TLS 1.3)
- Sauvegarde chiffrée (3-2-1 rule)
- Tokenisation des données sensibles

7. Couche Monitoring 👁

- SIEM (Security Information and Event Management)
- Alertes temps réel
- Audit logs centralisés
- Plan de réponse aux incidents

Exemple pratique (Application web) :

```
Attaquant
   ↓
1. WAF bloque patterns d'attaque connus
   ↓ (si bypass)
2. Load balancer avec rate limiting
   ↓ (si bypass)
3. Application valide et échappe les entrées
   ↓ (si bypass)
4. Requêtes préparées empêchent injection SQL
   ↓ (si bypass)
5. Compte BDD avec droits minimaux
   ↓ (si bypass)
6. Données sensibles chiffrées en BDD
   ↓ (si accès)
7. Monitoring détecte l'activité anormale → Alerte
```

Avantages :

- Pas de single point of failure
- Réduction de la surface d'attaque
- Temps de détection réduit
- Mitigation des zero-days

Bases de données et SQL {#sql}

1. Expliquez les différents types de jointures SQL

Réponse :

⚠ **RAPPEL IMPORTANT : INNER JOIN = JOIN** (strictement équivalents)

Types de jointures avec exemples visuels :

1. INNER JOIN (ou simplement JOIN) ➡ Seulement les lignes avec correspondance dans les deux tables

```
SELECT u.name, o.order_date, o.total
FROM users u
INNER JOIN orders o ON u.id = o.user_id;
```

Résultat : Uniquement les utilisateurs qui ont des commandes

2. LEFT JOIN (ou LEFT OUTER JOIN) ➡ Toutes les lignes de la table de gauche + correspondances de droite

```
SELECT u.name, o.order_date, o.total
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
```

Résultat :

- Tous les utilisateurs
- Commandes si elles existent
- NULL pour utilisateurs sans commande

3. RIGHT JOIN (ou RIGHT OUTER JOIN) ➡ Toutes les lignes de la table de droite + correspondances de gauche

```
SELECT u.name, o.order_date, o.total
FROM users u
RIGHT JOIN orders o ON u.id = o.user_id;
```

Résultat :

- Toutes les commandes
- Utilisateur associé si existe
- NULL si commande sans utilisateur (rare)

4. FULL OUTER JOIN ➡ Toutes les lignes des deux tables (union)

```
SELECT u.name, o.order_date, o.total
FROM users u
FULL OUTER JOIN orders o ON u.id = o.user_id;
```

Résultat : Utilisateurs sans commandes + Commandes sans utilisateurs + Correspondances

⚠ **Non supporté par MySQL** (utiliser `UNION` de LEFT et RIGHT)

5. CROSS JOIN ➡ Produit cartésien (toutes les combinaisons)

```
SELECT c.name, s.size
FROM colors c
CROSS JOIN sizes s;
```

Résultat : Si 3 couleurs × 4 tailles = 12 lignes

6. SELF JOIN ➡ Jointure d'une table avec elle-même

```
SELECT e1.name AS employee, e2.name AS manager
FROM employees e1
LEFT JOIN employees e2 ON e1.manager_id = e2.id;
```

Cas d'usage typiques :

Jointure	Quand l'utiliser
INNER	Données liées obligatoires (commandes avec clients)
LEFT	Inclure tous les enregistrements principaux (tous les clients, avec ou sans commandes)
RIGHT	Rarement utilisé (préférer LEFT avec tables inversées)
FULL	Comparaison de deux ensembles (audit de données)
CROSS	Générer des combinaisons (catalogue produits)

Performance :

- Toujours indexer les colonnes de jointure (FK)
- INNER JOIN généralement plus rapide que LEFT JOIN
- Limiter le nombre de jointures (max 5-6 pour performance)

2. Qu'est-ce que la normalisation de base de données ?

Réponse :

La normalisation organise les données pour **réduire la redondance** et **améliorer l'intégrité**.

Les 3 formes normales essentielles :

1NF (Première Forme Normale) ✅ Atomicité

Règles :

- Chaque cellule contient une **valeur atomique** (indivisible)
- Pas de listes ou de valeurs multiples
- Chaque enregistrement est unique (clé primaire)

× **Non 1NF :**

```
id | nom   | telephones
---|-------|------------------
1  | Alice | 0601..., 0602...
```

✅ **1NF :**

```
id | nom   | telephone
---|-------|----------
1  | Alice | 0601...
1  | Alice | 0602...
```

2NF (Deuxième Forme Normale) ✅ Dépendance fonctionnelle complète

Règles :

- Respecte 1NF
- Aucun attribut non-clé ne dépend d'une **partie** de la clé primaire
- Concerne les clés primaires composites

× **Non 2NF :**

```
commande_id | produit_id | quantite | prix_produit | nom_produit
```

(prix_produit et nom_produit dépendent uniquement de produit_id, pas de la clé composite)

✅ **2NF :**

```
Table COMMANDE_LIGNE:
commande_id | produit_id | quantite

Table PRODUIT:
produit_id | prix | nom
```

3NF (Troisième Forme Normale) ✅ Pas de dépendance transitive

Règles :

- Respecte 2NF
- Aucun attribut non-clé ne dépend d'un autre attribut non-clé

× **Non 3NF :**

```
employe_id | nom | code_postal | ville
```

(ville dépend de code_postal, pas directement de employe_id)

✅ **3NF :**

```
Table EMPLOYE:
employe_id | nom | code_postal

Table CODE_POSTAL:
code_postal | ville
```

Formes normales avancées (rarement nécessaires) :

- **BCNF** (Boyce-Codd) : 3NF renforcée
- **4NF** : Élimination des dépendances multivaluées
- **5NF** : Élimination des dépendances de jointure

Avantages de la normalisation :

- ✅ Pas de données dupliquées
- ✅ Mises à jour cohérentes (un seul endroit)
- ✅ Intégrité des données garantie
- ✅ Économie d'espace disque

Inconvénients :

- × Plus de jointures (impact performance)
- × Requêtes plus complexes
- × Peut nécessiter dénormalisation pour performances

Dénormalisation stratégique :

Pour les systèmes à forte lecture (analytics, reporting) :

- Duplication calculée (montant total commande)
- Tables agrégées pré-calculées
- Vues matérialisées

3. Différences entre SQL et NoSQL

Réponse :

Aspect	SQL (Relationnel)	NoSQL (Non-relationnel)
Schéma	Fixe, défini à l'avance	Flexible, évolutif

Aspect	SQL (Relationnel)	NoSQL (Non-relationnel)
Structure	Tables avec lignes/colonnes	Documents, Clés-valeurs, Colonnes, Graphes
Relations	Jointures (JOIN)	Dénormalisé ou références
ACID	Transactions complètes garanties	BASE (eventual consistency)
Scalabilité	Verticale (serveur plus puissant)	Horizontale (+ de serveurs)
Langage	SQL standardisé	APIs spécifiques
Consistance	Forte (forte cohérence)	Éventuelle (mais tunable)
Cas d'usage	Données structurées, transactions	Big Data, applications temps réel

Types de bases NoSQL :

1. Document (MongoDB, CouchDB)

```
{
  "_id": "123",
  "nom": "Alice",
  "adresses": [
    { "type": "domicile", "ville": "Paris" },
    { "type": "travail", "ville": "Lyon" }
  ],
  "commandes": [
    { "produit": "Laptop", "prix": 899 }
  ]
}
```

Avantages :

- Schéma flexible (champs variables)
- Mapping objet naturel (JSON)
- Requêtes imbriquées performantes

2. Clé-Valeur (Redis, DynamoDB)

```
user:123 → { "nom": "Alice", "email": "alice@example.com" }
session:abc → { "userId": 123, "expires": 1634567890 }
```

Avantages :

- Ultra-rapide (lecture/écriture)
- Cache distribué
- Simple à implémenter

3. Colonne (Cassandra, HBase)

```
Row Key: user_123
  Column Family: profile
    nom: Alice
    email: alice@example.com
  Column Family: activity
    last_login: 2024-01-15
    login_count: 42
```

Avantages :

- Optimisé pour write-heavy
- Scalabilité massive
- Compression efficace

4. Graphe (Neo4j, ArangoDB)

```
(Alice)-[:FRIEND_OF]->(Bob)
(Alice)-[:WORKS_AT]->(Company)
(Bob)-[:LIVES_IN]->(Paris)
```

Avantages :

- Relations complexes performantes
- Recommandations, réseaux sociaux
- Détection de fraude

Quand choisir SQL vs NoSQL :

Choisir SQL si :

- ✅ Données structurées et relationnelles
- ✅ Transactions complexes (banque, e-commerce)
- ✅ Requêtes analytiques (reporting)
- ✅ Cohérence forte obligatoire (ACID)
- ✅ Schéma stable et bien défini

Choisir NoSQL si :

- ✅ Volumes massifs de données (Big Data)
- ✅ Scalabilité horizontale nécessaire
- ✅ Schéma évolutif ou variable
- ✅ Temps réel / haute disponibilité prioritaire
- ✅ Données non structurées (logs, IoT)

Approche Polyglotte :

Utiliser plusieurs types de bases selon les besoins :

- **PostgreSQL** : Données métier transactionnelles

- **Redis** : Cache, sessions
- **Elasticsearch** : Recherche full-text
- **MongoDB** : Catalogues produits
- **Neo4j** : Graphe de relations

4. Comment optimiser les performances d'une base de données ?

Réponse :

1. Index (PRIORITÉ #1) 🚀

Types d'index :

```
-- Index simple
CREATE INDEX idx_user_email ON users(email);

-- Index composite (ordre important !)
CREATE INDEX idx_order_user_date ON orders(user_id, order_date);
-- Efficace pour : WHERE user_id = X AND order_date > Y
-- Moins efficace pour : WHERE order_date > Y seul

-- Index unique
CREATE UNIQUE INDEX idx_user_username ON users(username);

-- Index full-text
CREATE FULLTEXT INDEX idx_article_content ON articles(title, content);
```

Quand créer un index :

- ✅ Colonnes dans WHERE
- ✅ Colonnes dans JOIN
- ✅ Colonnes dans ORDER BY
- ✅ Clés étrangères (FK)

Quand éviter :

- ❌ Tables petites (< 1000 lignes)
- ❌ Colonnes rarement utilisées
- ❌ Colonnes avec peu de valeurs distinctes (genre: M/F)
- ❌ Tables avec beaucoup d'INSERT/UPDATE (coût maintenance)

2. Optimisation des requêtes 📊

```
-- ❌ MAL : SELECT *
SELECT * FROM users WHERE id = 123;

-- ✅ BIEN : Colonnes spécifiques
SELECT id, name, email FROM users WHERE id = 123;

-- ❌ MAL : OR (n'utilise pas les index)
```

```sql
SELECT * FROM products WHERE category = 'books' OR category = 'ebooks';

-- ✅ BIEN : IN
SELECT * FROM products WHERE category IN ('books', 'ebooks');

-- ❌ MAL : Fonction sur colonne indexée
SELECT * FROM orders WHERE YEAR(order_date) = 2024;

-- ✅ BIEN : Utilise l'index
SELECT * FROM orders
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01';

-- Limiter les résultats
SELECT * FROM products ORDER BY price DESC LIMIT 10;

-- Pagination efficace (keyset pagination)
SELECT * FROM products
WHERE id > 1000  -- dernier ID vu
ORDER BY id LIMIT 20;
```

3. EXPLAIN / EXPLAIN ANALYZE 🔍

```sql
EXPLAIN ANALYZE
SELECT u.name, COUNT(o.id)
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.id;
```

Indicateurs à surveiller :

- **type: ALL** ❌ = Full table scan (ajouter index)
- **type: index** ⚠️ = Index scan (correct mais peut être amélioré)
- **type: ref** ✅ = Bon (utilise index)
- **type: const** ✅✅ = Optimal (clé primaire)
- **rows** : Nombre de lignes examinées (plus bas = mieux)
- **Extra: Using filesort** ⚠️ = Tri en mémoire (ajouter index sur ORDER BY)
- **Extra: Using temporary** ⚠️ = Table temporaire (GROUP BY non indexé)

4. Partitioning (Partitionnement) 📦

```sql
-- Partition par plage de dates
CREATE TABLE orders (
  id INT,
  user_id INT,
  order_date DATE,
  total DECIMAL(10,2)
)
PARTITION BY RANGE (YEAR(order_date)) (
  PARTITION p2022 VALUES LESS THAN (2023),
```

```
  PARTITION p2023 VALUES LESS THAN (2024),
  PARTITION p2024 VALUES LESS THAN (2025),
  PARTITION pmax VALUES LESS THAN MAXVALUE
);
```

Avantages :

- Requêtes plus rapides (partition pruning)
- Maintenance facilitée (purge ancienne partition)
- Meilleure parallélisation

5. Caching (Mise en cache) ⚡

Niveaux de cache :

```
Application
    ↓
Cache Applicatif (Redis)
    ↓ (cache miss)
Cache Requêtes (MySQL Query Cache - deprecated)
    ↓ (cache miss)
Base de données
```

Exemple avec Redis :

```
async function getUser(id) {
  // 1. Vérifier le cache
  const cached = await redis.get(`user:${id}`);
  if (cached) return JSON.parse(cached);

  // 2. Si absent, requête BDD
  const user = await db.query('SELECT * FROM users WHERE id = ?', [id]);

  // 3. Mettre en cache (TTL 1 heure)
  await redis.setex(`user:${id}`, 3600, JSON.stringify(user));

  return user;
}
```

Stratégies de cache :

- **Cache-aside** : Application gère le cache
- **Write-through** : Écriture BDD + cache simultanément
- **Write-behind** : Cache d'abord, BDD asynchrone

6. Connection Pooling 🏊

```
// Node.js avec pool de connexions
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'mydb',
  connectionLimit: 10,  // Max 10 connexions simultanées
  queueLimit: 0
});
```

Avantages :

- Réutilisation des connexions (pas de création/fermeture répétée)
- Limite la charge sur le serveur BDD
- Améliore les temps de réponse (50-70%)

7. Dénormalisation ciblée 📈

```
-- Au lieu de JOIN systématique
SELECT o.*, u.name, u.email
FROM orders o
JOIN users u ON o.user_id = u.id;

-- Dupliquer les données fréquemment lues
ALTER TABLE orders
ADD COLUMN user_name VARCHAR(255),
ADD COLUMN user_email VARCHAR(255);
```

⚠ **Compromis :** Espace disque vs Performance lecture

8. Monitoring continu 📊

Métriques clés :

- **Slow Query Log** : Requêtes > 2 secondes
- **Connexions actives** : Surveiller les pics
- **Buffer Pool Hit Rate** : > 99% (cache mémoire efficace)
- **Table/Index size** : Croissance anormale

Outils :

- **MySQL Workbench** : Performance Dashboard
- **pg_stat_statements** (PostgreSQL) : Stats requêtes
- **New Relic / DataDog** : Monitoring APM
- **Percona Toolkit** : Analyse performance

Checklist optimisation :

1. ✅ Index sur colonnes WHERE/JOIN/ORDER BY

2. ✅ SELECT uniquement colonnes nécessaires
3. ✅ EXPLAIN pour comprendre le plan d'exécution
4. ✅ Limiter résultats avec LIMIT
5. ✅ Éviter SELECT * et requêtes N+1
6. ✅ Connection pooling activé
7. ✅ Cache (Redis) pour données fréquentes
8. ✅ Partitioning si tables > 100M lignes
9. ✅ Monitoring et alertes configurés

5. Différence entre MCD, MLD et MPD

Réponse :

Progression de la conception : Concept → Logique → Physique

MCD (Modèle Conceptuel de Données) 🧠 Niveau MÉTIER

Objectif : Représenter les données du point de vue métier, indépendamment de toute technologie.

Éléments :

- **Entités** : Objets métier (Client, Commande, Produit)
- **Associations** : Relations entre entités (PASSER, CONTENIR)
- **Cardinalités** : (min, max) pour chaque côté de l'association
- **Attributs** : Propriétés des entités (nom, prénom, date)

Exemple MCD :

```
CLIENT (id_client, nom, prenom, email)
    |
    | (1,n) PASSER (0,n)
    |
COMMANDE (id_commande, date, montant_total)
    |
    | (1,n) CONTENIR (0,n)
    |
PRODUIT (id_produit, libelle, prix)
```

Cardinalités expliquées :

- `(1,n)` CLIENT-PASSER : Un client passe **minimum 1, maximum n** commandes
- `(0,n)` PASSER-COMMANDE : Une commande est passée par **0 ou plusieurs** clients (si commande invité possible)

MLD (Modèle Logique de Données) 📋 Niveau LOGIQUE

Objectif : Transformer le MCD en tables avec clés primaires/étrangères, sans détails techniques.

Transformation MCD → MLD :

Règle 1 : Entités → Tables

```
CLIENT (id_client, nom, prenom, email)
         PK
```

Règle 2 : Associations selon cardinalités

Cardinalité	Transformation
(1,1) - (1,1)	Clé étrangère dans n'importe quelle table
(1,1) - (0,n)	Clé étrangère côté (0,n)
(0,n) - (0,n)	Table de jonction avec 2 FK

Exemple MLD :

```
CLIENT (id_client PK, nom, prenom, email)

COMMANDE (id_commande PK, date, montant_total, #id_client FK)
                                                    ↑
                                          Clé étrangère

PRODUIT (id_produit PK, libelle, prix)

LIGNE_COMMANDE (#id_commande FK, #id_produit FK, quantite, prix_unitaire)
                PK composite : (id_commande, id_produit)
```

MPD (Modèle Physique de Données) 💾 Niveau IMPLÉMENTATION

Objectif : Implémentation SQL concrète avec types de données, contraintes, index.

Différences avec MLD :

- **Types de données précis** (VARCHAR(255), DECIMAL(10,2), DATETIME)
- **Contraintes** (NOT NULL, UNIQUE, CHECK, DEFAULT)
- **Index** (PRIMARY KEY, FOREIGN KEY, INDEX)
- **Triggers, procédures stockées**
- **Optimisations spécifiques** au SGBD (MySQL, PostgreSQL, etc.)

Exemple MPD (SQL) :

```sql
CREATE TABLE CLIENT (
  id_client INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  date_inscription DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```sql
CREATE TABLE COMMANDE (
  id_commande INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  date_commande DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  montant_total DECIMAL(10,2) NOT NULL CHECK (montant_total >= 0),
  id_client INT UNSIGNED NOT NULL,

  FOREIGN KEY (id_client) REFERENCES CLIENT(id_client)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,

  INDEX idx_client (id_client),
  INDEX idx_date (date_commande)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE PRODUIT (
  id_produit INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  libelle VARCHAR(255) NOT NULL,
  prix DECIMAL(8,2) NOT NULL CHECK (prix > 0),
  stock INT UNSIGNED DEFAULT 0,
  actif BOOLEAN DEFAULT TRUE,

  INDEX idx_actif (actif)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE LIGNE_COMMANDE (
  id_commande INT UNSIGNED,
  id_produit INT UNSIGNED,
  quantite INT UNSIGNED NOT NULL CHECK (quantite > 0),
  prix_unitaire DECIMAL(8,2) NOT NULL,

  PRIMARY KEY (id_commande, id_produit),

  FOREIGN KEY (id_commande) REFERENCES COMMANDE(id_commande)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  FOREIGN KEY (id_produit) REFERENCES PRODUIT(id_produit)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Comparaison synthétique :

Aspect	MCD	MLD	MPD
Niveau	Conceptuel	Logique	Physique
Public	Métier/Client	Analyste	Développeur/DBA
Notation	Entités, Associations	Tables, FK	SQL complet
Types de données	Non	Non	Oui (VARCHAR, INT...)

Aspect	MCD	MLD	MPD
Contraintes	Cardinalités	Clés PK/FK	CHECK, DEFAULT, UNIQUE...
Optimisation	Non	Non	Index, Engine, Partitions
Indépendance tech	Totale	Totale	Spécifique SGBD

Différence MLD/MPD (PIÈGE FRÉQUENT) :

La différence principale est le **niveau de détail technique** :

- **MLD** : Structure logique (quelles tables, quelles relations)
- **MPD** : Implémentation concrète (comment le créer en SQL)

Architecture et conception {#architecture}

1. Expliquez le pattern MVC (Model-View-Controller)

Réponse :

MVC sépare une application en 3 composants interconnectés.

Architecture MVC :

```
        USER
          ↓ (1. Interaction)
        VIEW ←----------┐
          ↓ (2. Route)  | (6. Rendu)
      CONTROLLER        |
        ↓     ↑         |
    (3) |     | (5)     |
        ↓     ↑         |
        MODEL ──────────┘
          ↓ (4. Query/Update)
        DATABASE
```

Responsabilités de chaque composant :

1. Model (Modèle) 💾 **Rôle :** Gestion des données et logique métier

- Accès base de données (CRUD)
- Règles métier (validation, calculs)
- État de l'application
- Notifications des changements (Observer pattern)

```php
// Symfony Entity (Model)
class Product {
  private int $id;
  private string $name;
```

```
  private float $price;

  public function calculateTotalWithTax(float $taxRate): float {
    return $this->price * (1 + $taxRate);  // Logique métier
  }

  public function isAvailable(): bool {
    return $this->stock > 0;
  }
}
```

2. View (Vue) 🎨 **Rôle :** Présentation des données à l'utilisateur

- Templates HTML
- Affichage des données du Model
- Capture des interactions utilisateur
- **Aucune logique métier** (seulement présentation)

```
<!-- Twig template (View) -->
<div class="product">
  <h2>{{ product.name }}</h2>
  <p>Prix: {{ product.price|number_format(2) }} €</p>

  {% if product.isAvailable() %}
    <button>Ajouter au panier</button>
  {% else %}
    <span class="out-of-stock">Rupture de stock</span>
  {% endif %}
</div>
```

3. Controller (Contrôleur) 🎮 **Rôle :** Chef d'orchestre, fait le lien Model ↔ View

- Reçoit les requêtes utilisateur
- Appelle le Model pour récupérer/modifier données
- Sélectionne la View appropriée
- Passe les données à la View

```
// Symfony Controller
class ProductController extends AbstractController {
  #[Route('/product/{id}')]
  public function show(int $id, ProductRepository $repo): Response {
    // 1. Récupérer données du Model
    $product = $repo->find($id);

    // 2. Traiter logique
    if (!$product) {
      throw $this->createNotFoundException();
    }
```

```
      // 3. Passer à la View et renvoyer réponse
      return $this->render('product/show.html.twig', [
        'product' => $product
      ]);
    }
  }
```

Flux complet (exemple e-commerce) :

Scénario : Utilisateur ajoute un produit au panier

1. **User clique** "Ajouter au panier" (View)
2. **View** envoie requête POST `/cart/add?product=123` (Controller)
3. **Controller** reçoit la requête
4. **Controller** → **Model** : `cart->addProduct(123)`
5. **Model** récupère le produit, vérifie stock, ajoute au panier
6. **Model** → **Controller** : Confirmation ajout
7. **Controller** → **View** : Données du panier mis à jour
8. **View** affiche panier avec nouveau produit

Avantages du MVC :

✅ **Séparation des responsabilités (SoC)**

- Chaque composant a un rôle clair
- Modifications isolées (CSS ≠ logique métier)

✅ **Testabilité**

- Model testable indépendamment
- Mocking facile des dépendances

✅ **Développement parallèle**

- Designer travaille sur View
- Développeur backend sur Model
- Développeur frontend sur Controller

✅ **Réutilisabilité**

- Même Model pour web + mobile + API
- Views multiples pour un même Model

✅ **Maintenabilité**

- Code organisé et prévisible
- Facilite l'onboarding nouveaux développeurs

Variantes du MVC :

- **MVP** (Model-View-Presenter) : Presenter remplace Controller, plus de logique
- **MVVM** (Model-View-ViewModel) : ViewModel avec data-binding (Angular)

- **MVT** (Model-View-Template) : Variante Django
- **MVC moderne** : Souvent combiné avec architecture en couches/hexagonale

Frameworks utilisant MVC :

- **Backend** : Symfony, Spring MVC, Laravel, ASP.NET MVC, Ruby on Rails
- **Frontend** : Angular (MVVM), Backbone.js

2. Qu'est-ce qu'une architecture microservices ?

Réponse :

Architecture décomposant une application en **services indépendants et spécialisés**.

Principe fondamental :

```
MONOLITHE                    MICROSERVICES
┌──────────────┐             ┌─────────┐  ┌─────────┐  ┌─────────┐
│              │             │ User    │  │Product  │  │ Order   │
│ Application  │    →        │Service  │  │Service  │  │Service  │
│  complète    │             │         │  │         │  │         │
│              │             └────┬────┘  └────┬────┘  └────┬────┘
└──────────────┘                  │            │            │
       │                          ┌────────────┴────────────┴────┐
   ┌───┴───┐                      │    API Gateway / Service     │
   │  BDD  │                      │    Mesh (Istio/Consul)       │
   └───────┘                      └──────────────────────────────┘
```

Caractéristiques essentielles :

1. Services autonomes 🏗️

- Chaque service = fonctionnalité métier spécifique
- Base de données dédiée (database per service)
- Déployé indépendamment
- Équipe dédiée (vous construisez, vous gérez)

Exemple e-commerce :

```
- user-service        → Gestion utilisateurs (PostgreSQL)
- product-service     → Catalogue produits (MongoDB)
- order-service       → Commandes (PostgreSQL)
- payment-service     → Paiements (PostgreSQL + Redis)
- inventory-service   → Stocks (MySQL)
- notification-service → Emails/SMS (Redis queue)
- search-service      → Recherche (Elasticsearch)
```

2. Communication inter-services 🔄

Synchrone (REST/gRPC) :

```
// order-service appelle product-service
const product = await axios.get('http://product-service/api/products/123');
```

Asynchrone (Message Queue) :

```
// order-service publie événement
await messageQueue.publish('order.created', {
  orderId: 123,
  userId: 456
});

// notification-service écoute
messageQueue.subscribe('order.created', async (data) => {
  await sendEmail(data.userId, 'Commande confirmée');
});
```

Technologies :

- **RabbitMQ, Apache Kafka** : Message brokers
- **gRPC** : Communication binaire performante
- **REST** : Standard HTTP classique

3. Orchestration/Choreography 🎼

Orchestration (centralisée) :

```
Order Service (orchestrateur)
    ├─> Product Service (vérifier stock)
    ├─> Payment Service (payer)
    ├─> Inventory Service (réserver)
    └─> Notification Service (confirmer)
```

Choreography (événements) :

```
Order Created → Product Service écoute → Stock vérifié
              → Payment Service écoute → Paiement traité
              → Inventory Service écoute → Stock mis à jour
              → Notification Service écoute → Email envoyé
```

Avantages des microservices :

✅ **Scalabilité fine**

- Scaler uniquement les services sous charge
- Exemple : Black Friday → Scaler product-service + order-service uniquement

✅ **Indépendance technologique**

- Chaque service peut utiliser la stack optimale
- User-service : Node.js + PostgreSQL
- Product-service : Python + MongoDB
- Search-service : Java + Elasticsearch

✅ **Déploiement indépendant**

- Déployer order-service sans toucher product-service
- Réduction du risque (blast radius limité)
- CI/CD par service

✅ **Équipes autonomes**

- Ownership clair
- Développement parallèle
- Moins de coordination requise

✅ **Résilience**

- Panne isolée (product-service down ≠ tout down)
- Circuit breaker pattern
- Graceful degradation

Inconvénients et défis :

× **Complexité opérationnelle**

- 10 microservices = 10× monitoring, logs, déploiements
- Nécessite DevOps mature

× **Consistance des données**

- Pas de transactions ACID cross-services
- Eventual consistency
- Saga pattern pour transactions distribuées

× **Communication réseau**

- Latence (appels inter-services)
- Gestion des pannes réseau
- Besoin de retry, timeout, circuit breaker

× **Gestion des données distribuées**

- Jointures impossibles
- Agrégation complexe
- CQRS/Event Sourcing souvent nécessaire

× **Testing plus complexe**

- Tests d'intégration multi-services
- Contract testing (Pact)
- Environnements de test coûteux

Patterns microservices essentiels :

1. API Gateway 🚪

```
Client
  ↓
API Gateway (authentification, rate limiting, routing)
  ├─> User Service
  ├─> Product Service
  └─> Order Service
```

2. Service Discovery 🔍

- Consul, Eureka, Kubernetes DNS
- Services s'enregistrent automatiquement
- Discovery dynamique des instances

3. Circuit Breaker ⚡

```
// Si product-service down, ne pas attendre
const product = await circuitBreaker.execute(
  () => axios.get('http://product-service/products/123'),
  { timeout: 2000, fallback: { name: 'Indisponible' } }
);
```

4. Event Sourcing 📜

- Stocker les événements plutôt que l'état
- Reconstituer l'état en rejouant les événements
- Audit trail complet

5. CQRS (Command Query Responsibility Segregation) 📊

- Séparation lecture/écriture
- Optimisation indépendante
- Base d'écriture ≠ base de lecture

Quand utiliser les microservices :

✅ **OUI si :**

- Application complexe et large
- Équipes multiples (> 20 développeurs)

- Besoin de scalabilité différenciée
- Cycles de release indépendants souhaités
- Maturité DevOps élevée

× **NON si :**

- Petite application / MVP
- Équipe < 5 développeurs
- Pas d'expertise DevOps
- Monolithe modulaire suffit

Alternative : Monolithe modulaire

- Découpage en modules bien définis
- Base de code unique
- Migration vers microservices si nécessaire

3. Expliquez les principes SOLID

Réponse :

SOLID = 5 principes de conception orientée objet pour un code maintenable.

S - Single Responsibility Principle 🎯 **Principe :** Une classe ne doit avoir qu'une seule raison de changer.

× **Violation SRP :**

```
class User {
  saveToDatabase() { /* ... */ }      // Responsabilité 1: Persistance
  sendEmail() { /* ... */ }            // Responsabilité 2: Notification
  generateReport() { /* ... */ }       // Responsabilité 3: Reporting
}
```

✅ **Respect SRP :**

```
class User {
  // Données métier uniquement
  constructor(public name: string, public email: string) {}
}

class UserRepository {
  save(user: User) { /* BDD */ }
}

class EmailService {
  send(user: User, message: string) { /* Email */ }
}

class ReportGenerator {
```

```
  generate(users: User[]) { /* PDF */ }
}
```

Bénéfices :

- Changement de BDD → Modifier UserRepository uniquement
- Changement provider email → Modifier EmailService uniquement
- Tests plus faciles (mocking précis)

O - Open/Closed Principle 🔓 Principe : Ouvert à l'extension, fermé à la modification.

× **Violation OCP :**

```
class PaymentProcessor {
  process(payment: Payment) {
    if (payment.type === 'creditcard') {
      // Logique carte bancaire
    } else if (payment.type === 'paypal') {
      // Logique PayPal
    } else if (payment.type === 'crypto') {  // ← Modification du code
existant
      // Logique crypto
    }
  }
}
```

✅ **Respect OCP (avec abstraction) :**

```
interface PaymentMethod {
  process(amount: number): boolean;
}

class CreditCardPayment implements PaymentMethod {
  process(amount: number): boolean {
    // Logique carte bancaire
    return true;
  }
}

class PayPalPayment implements PaymentMethod {
  process(amount: number): boolean {
    // Logique PayPal
    return true;
  }
}

// Extension sans modification
class CryptoPayment implements PaymentMethod {  // ← Nouvelle classe, pas
```

```
de modif
  process(amount: number): boolean {
    // Logique crypto
    return true;
  }
}

class PaymentProcessor {
  constructor(private method: PaymentMethod) {}

  execute(amount: number) {
    return this.method.process(amount);
  }
}
```

Bénéfices :

- Ajouter crypto sans toucher au code existant
- Pas de régression possible
- Tests existants toujours valides

L - Liskov Substitution Principle 🔄 **Principe :** Les objets d'une classe dérivée doivent pouvoir remplacer les objets de la classe de base sans altérer le comportement.

× **Violation LSP :**

```
class Bird {
  fly() { console.log('Flying'); }
}

class Penguin extends Bird {
  fly() {
    throw new Error('Penguins cannot fly!');  // ← Violation !
  }
}

function makeBirdFly(bird: Bird) {
  bird.fly();  // Crash si bird est un Penguin
}
```

✅ **Respect LSP :**

```
interface Bird {
  move(): void;
}

class FlyingBird implements Bird {
  move() { this.fly(); }
```

```
  fly() { console.log('Flying'); }
}

class Penguin implements Bird {
  move() { this.swim(); }
  swim() { console.log('Swimming'); }
}

function moveBird(bird: Bird) {
  bird.move();  // ✅ Fonctionne pour tous les types de Bird
}
```

Bénéfices :

- Polymorphisme fiable
- Pas de surprises au runtime
- Contrats respectés

I - Interface Segregation Principle 🔪 **Principe :** Ne pas forcer une classe à implémenter des interfaces qu'elle n'utilise pas.

× **Violation ISP (interface trop large) :**

```
interface Worker {
  work(): void;
  eat(): void;
  sleep(): void;
}

class Robot implements Worker {
  work() { /* OK */ }
  eat() { /* ??? Robot ne mange pas */ }
  sleep() { /* ??? Robot ne dort pas */ }
}
```

✅ **Respect ISP (interfaces spécifiques) :**

```
interface Workable {
  work(): void;
}

interface Eatable {
  eat(): void;
}

interface Sleepable {
  sleep(): void;
}
```

```
class Human implements Workable, Eatable, Sleepable {
  work() { /* ... */ }
  eat() { /* ... */ }
  sleep() { /* ... */ }
}

class Robot implements Workable {
  work() { /* ... */ }
  // Pas besoin d'implémenter eat() et sleep()
}
```

Bénéfices :

- Interfaces ciblées et cohérentes
- Pas de méthodes vides ou exceptions
- Couplage réduit

D - Dependency Inversion Principle ⬆ Principe :

1. Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau. Les deux doivent dépendre d'abstractions.
2. Les abstractions ne doivent pas dépendre des détails. Les détails doivent dépendre des abstractions.

× **Violation DIP (dépendance concrète) :**

```
class MySQLDatabase {
  save(data: any) { /* MySQL specific */ }
}

class UserService {
  private db = new MySQLDatabase();  // ← Couplage fort

  saveUser(user: User) {
    this.db.save(user);  // Impossible de changer de BDD
  }
}
```

✅ **Respect DIP (dépendance abstraction) :**

```
// Abstraction
interface Database {
  save(data: any): void;
}

// Implémentations concrètes
class MySQLDatabase implements Database {
  save(data: any) { /* MySQL */ }
```

```
}

class MongoDatabase implements Database {
  save(data: any) { /* MongoDB */ }
}

// Service dépend de l'abstraction
class UserService {
  constructor(private db: Database) {}  // ← Injection de dépendance

  saveUser(user: User) {
    this.db.save(user);  // ✅ Fonctionne avec n'importe quelle BDD
  }
}

// Utilisation
const service1 = new UserService(new MySQLDatabase());
const service2 = new UserService(new MongoDatabase());
```

Bénéfices :

- Changement de technologie sans impact
- Testabilité (mock facile)
- Flexibilité maximale

Injection de dépendances (DI) :

```
// Angular
@Injectable()
export class UserService {
  constructor(private http: HttpClient) {}  // DI automatique
}

// Symfony
class UserController {
  public function __construct(
    private UserRepository $repo  // DI automatique
  ) {}
}
```

Résumé SOLID :

Principe	Question clé	Bénéfice principal
S	Une classe a-t-elle une seule responsabilité ?	Cohésion
O	Puis-je étendre sans modifier ?	Évolutivité
L	Les sous-classes sont-elles substituables ?	Polymorphisme fiable

Principe	Question clé	Bénéfice principal
I	Les interfaces sont-elles ciblées ?	Couplage faible
D	Dépends-je d'abstractions ?	Flexibilité

Bénéfices globaux :

- ✅ Code maintenable et évolutif
- ✅ Réduction du couplage
- ✅ Facilite les tests unitaires
- ✅ Améliore la réutilisabilité
- ✅ Moins de bugs en production

4. Qu'est-ce que l'architecture hexagonale ?

Réponse :

Architecture hexagonale (Ports & Adapters) isole la logique métier des détails techniques.

Concept central : Le domaine métier ne doit dépendre de RIEN (ni BDD, ni framework, ni UI).

Structure :

```
┌──────────────────────────────────────┐
│      ADAPTATEURS PRIMAIRES           │
│   (Entrées = Drivers)                │
│                                      │
│   ┌─────┐   ┌─────┐   ┌─────┐       │
│   │ API │   │ CLI │   │ Web │       │
│   │REST │   │     │   │ UI  │       │
│   └─────┘   └─────┘   └─────┘       │
└──────┬──────┴──────┴──────┴──────┘
       │        │        │
┌──────▼────────▼────────▼──────┐
│         PORTS (Interfaces)          │
│                                     │
│  UserService, ProductService        │
└──────────────────────────────────┘
                │
┌──────────────▼──────────────────┐
│      🔷 DOMAINE MÉTIER 🔷          │
│    (Logique applicative)            │
│                                     │
│  Entités, Cas d'usage,             │
│  Règles métier                      │
│                                     │
│  ⚠  AUCUNE DÉPENDANCE              │
│  vers framework/BDD/UI              │
└──────────────────────────────────┘
                │
┌──────────────▼──────────────────┐
```

```
                  │         PORTS (Interfaces)      │
                  │                                 │
                  │  UserRepository, EmailSender    │
                  └─────────┬─────────┬─────────────┘
                            │         │         │
                  ┌─────────▼─────────▼─────────▼─────────┐
                  │    ADAPTATEURS SECONDAIRES            │
                  │    (Sorties = Driven)                 │
                  │                                       │
                  │  ┌──────┐  ┌──────┐  ┌──────┐       │
                  │  │MySQL │  │ SMTP │  │Redis │       │
                  │  │ BDD  │  │Email │  │Cache │       │
                  │  └──────┘  └──────┘  └──────┘       │
                  └───────────────────────────────────────┘
```

Composants :

1. Domaine (Hexagone central) 🔷

- **Entités** : Objets métier purs
- **Value Objects** : Objets immuables (Email, Money)
- **Use Cases** : Scénarios métier
- **Business Rules** : Règles de validation

```
// Entité pure (domaine)
class Order {
  constructor(
    private id: string,
    private items: OrderItem[],
    private status: OrderStatus
  ) {}

  // Logique métier pure
  calculateTotal(): Money {
    return this.items.reduce(
      (sum, item) => sum.add(item.price),
      Money.zero()
    );
  }

  canBeCancelled(): boolean {
    return this.status === OrderStatus.PENDING;
  }
}
```

2. Ports (Interfaces) 🔌

Ports primaires (entrées) :

- Interfaces pour piloter l'application

- Exposées par le domaine

```
// Port primaire (USE CASE)
interface CreateOrderUseCase {
  execute(request: CreateOrderRequest): Promise<Order>;
}
```

Ports secondaires (sorties) :

- Interfaces pour accéder aux ressources externes
- Définies par le domaine, implémentées par les adaptateurs

```
// Port secondaire (REPOSITORY)
interface OrderRepository {
  save(order: Order): Promise<void>;
  findById(id: string): Promise<Order | null>;
}

// Port secondaire (SERVICE EXTERNE)
interface PaymentGateway {
  charge(amount: Money): Promise<PaymentResult>;
}
```

3. Adaptateurs primaires (Drivers) 📥

Pilotent l'application via les ports primaires.

```
// Adaptateur REST
@Controller('/orders')
export class OrderController {
  constructor(private createOrder: CreateOrderUseCase) {}

  @Post()
  async create(@Body() dto: CreateOrderDto) {
    const result = await this.createOrder.execute({
      customerId: dto.customerId,
      items: dto.items
    });
    return result;
  }
}

// Adaptateur CLI
class OrderCLI {
  constructor(private createOrder: CreateOrderUseCase) {}

  async run(args: string[]) {
    await this.createOrder.execute(/* ... */);
```

```
  }
}
```

4. Adaptateurs secondaires (Driven) 📤

Implémentent les ports secondaires.

```
// Adaptateur MySQL
class MySQLOrderRepository implements OrderRepository {
  constructor(private db: Database) {}

  async save(order: Order): Promise<void> {
    await this.db.query(
      'INSERT INTO orders (id, total, status) VALUES (?, ?, ?)',
      [order.id, order.total, order.status]
    );
  }

  async findById(id: string): Promise<Order | null> {
    const row = await this.db.queryOne('SELECT * FROM orders WHERE id = ?',
[id]);
    return row ? this.mapToOrder(row) : null;
  }
}

// Adaptateur Stripe Payment
class StripePaymentGateway implements PaymentGateway {
  async charge(amount: Money): Promise<PaymentResult> {
    const stripe = new Stripe(process.env.STRIPE_KEY);
    const charge = await stripe.charges.create({
      amount: amount.cents,
      currency: 'eur'
    });
    return { success: charge.status === 'succeeded' };
  }
}
```

Flux complet (exemple) :

Scénario : Créer une commande via API REST

```
1. HTTP POST /orders { customerId: 123, items: [...] }
        ↓
2. OrderController (adaptateur primaire REST)
        ↓
3. CreateOrderUseCase.execute() (port primaire)
        ↓
4. Order.calculateTotal() (domaine)
        ↓
5. PaymentGateway.charge() (port secondaire)
```

```
        ↓
6. StripePaymentGateway.charge() (adaptateur secondaire)
        ↓
7. OrderRepository.save() (port secondaire)
        ↓
8. MySQLOrderRepository.save() (adaptateur secondaire)
        ↓
9. Retour Order au Controller
        ↓
10. HTTP 201 Created
```

Avantages de l'architecture hexagonale :

✅ Indépendance technologique

- Changer MySQL → PostgreSQL : Créer nouvel adaptateur
- Changer Stripe → PayPal : Créer nouvel adaptateur
- Le domaine n'est JAMAIS modifié

✅ Testabilité maximale

```
// Test du domaine sans BDD ni API
describe('CreateOrderUseCase', () => {
  it('should create order successfully', async () => {
    // Mocks (adaptateurs en mémoire)
    const fakeRepo = new InMemoryOrderRepository();
    const fakePayment = new FakePaymentGateway();

    const useCase = new CreateOrderUseCase(fakeRepo, fakePayment);

    const order = await useCase.execute({ ... });

    expect(order.total).toBe(100);
  });
});
```

✅ Évolutivité

- Ajouter GraphQL en plus de REST : Nouvel adaptateur primaire
- Ajouter cache Redis : Nouvel adaptateur secondaire
- Domaine inchangé

✅ Compréhension métier

- Logique métier isolée et lisible
- Pas pollué par les détails techniques
- Domain-Driven Design (DDD) friendly

Quand utiliser l'architecture hexagonale :

✅ **OUI si :**

- Logique métier complexe
- Évolution technologique probable
- Testabilité prioritaire
- Équipe mature

× **NON si :**

- Application CRUD simple
- Équipe junior
- MVP rapide
- Overhead trop important

Comparaison avec autres architectures :

Architecture	Complexité	Testabilité	Indépendance tech
MVC classique	Faible	Moyenne	Faible
Architecture en couches	Moyenne	Bonne	Moyenne
Hexagonale	Élevée	Excellente	Totale
Clean Architecture	Très élevée	Excellente	Totale

Architecture hexagonale ≈ Clean Architecture (Robert C. Martin)

- Mêmes principes
- Vocabulaire différent
- Clean Architecture = Généralisation d'hexagonale

Tests et qualité {#tests}

1. Quels sont les différents types de tests ?

Réponse :

Pyramide des tests (de bas en haut) :

```
        /\
       /  \          🔺 Tests E2E (End-to-End)
      /    \        Peu nombreux, lents, coûteux
     /------\       Interface utilisateur complète
    /        \
   /          \      🔺 Tests d'Intégration
  /            \    Moyennement nombreux, moyennement rapides
 /--------------\   Interaction entre composants
/                \
                  \  🔺 Tests Unitaires
```

```
  /──────────────────────\ Très nombreux, rapides, peu coûteux
                           Fonction/méthode isolée
```

Types de tests par niveau :

1. Tests Unitaires 🧩 **Objectif :** Tester une fonction/méthode isolée

```
// Exemple Jest
describe('calculateTotal', () => {
  it('should calculate total with tax', () => {
    const cart = new ShoppingCart();
    cart.addItem({ price: 100, quantity: 2 });

    expect(cart.calculateTotal(0.2)).toBe(240);  // 200 + 20% TVA
  });

  it('should return 0 for empty cart', () => {
    const cart = new ShoppingCart();
    expect(cart.calculateTotal(0.2)).toBe(0);
  });
});
```

Caractéristiques :

- ✅ Ultra-rapides (millisecondes)
- ✅ Isolés (pas de BDD, pas d'API)
- ✅ Nombreux (70-80% de la suite de tests)
- ✅ Tests FIRST (Fast, Independent, Repeatable, Self-validating, Timely)

2. Tests d'Intégration 🔗 **Objectif :** Tester l'interaction entre composants

```
// Test API + BDD
describe('POST /api/users', () => {
  beforeEach(async () => {
    await database.clean();  // Base de test
  });

  it('should create user in database', async () => {
    const response = await request(app)
      .post('/api/users')
      .send({ name: 'Alice', email: 'alice@test.com' });

    expect(response.status).toBe(201);

    const userInDb = await database.query('SELECT * FROM users WHERE email
= ?',
      ['alice@test.com']);
    expect(userInDb).toBeDefined();
```

```
  });
});
```

Caractéristiques :

- ⚡ Moyennement rapides (secondes)
- 🔌 Dépendances réelles (BDD de test, services externes mockés)
- 📊 15-20% de la suite de tests

3. Tests Système/Fonctionnels 🌐 **Objectif :** Vérifier le système complet

```
// Exemple Cypress
describe('User Registration Flow', () => {
  it('should register and login user', () => {
    cy.visit('/register');
    cy.get('input[name="email"]').type('user@test.com');
    cy.get('input[name="password"]').type('SecurePass123!');
    cy.get('button[type="submit"]').click();

    cy.url().should('include', '/dashboard');
    cy.contains('Welcome, user@test.com');
  });
});
```

4. Tests d'Acceptation 👤 **Objectif :** Validation par le client/métier

```
# Gherkin (Cucumber)
Feature: Shopping Cart
  As a customer
  I want to add products to my cart
  So that I can purchase them later

  Scenario: Add product to cart
    Given I am on the product page
    And the product "Laptop" is available
    When I click "Add to cart"
    Then I should see "Product added to cart"
    And the cart total should be "1 item"
```

Types de tests par objectif :

Tests de Régression 🔄

- Vérifier qu'une modification n'a pas cassé l'existant
- Exécutés automatiquement après chaque déploiement
- Suite de tests automatisés critiques

Tests de Performance ⚡

- **Load Testing** : Comportement sous charge normale
- **Stress Testing** : Limite avant défaillance
- **Spike Testing** : Pics soudains de charge
- **Endurance Testing** : Comportement sur longue durée

```
// Exemple K6
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 },    // Monter à 100 users
    { duration: '5m', target: 100 },    // Rester à 100 users
    { duration: '2m', target: 0 },      // Redescendre à 0
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],   // 95% des requêtes < 500ms
  },
};

export default function () {
  const res = http.get('https://api.example.com/products');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 500ms': (r) => r.timings.duration < 500,
  });
}
```

Tests de Sécurité 🔒

- **SAST** (Static Application Security Testing) : Analyse code source
- **DAST** (Dynamic Application Security Testing) : Tests en cours d'exécution
- **Penetration Testing** : Simulation d'attaques
- **Dependency Scanning** : Vulnérabilités dépendances

Outils :

- **OWASP ZAP** : Scanner vulnérabilités web
- **Snyk** : Analyse dépendances
- **SonarQube** : Quality gate + sécurité

Tests de Compatibilité 📱

- Cross-browser (Chrome, Firefox, Safari, Edge)
- Cross-device (Desktop, Tablet, Mobile)
- Cross-OS (Windows, Mac, Linux)

Outils :

- **BrowserStack** : Navigateurs réels cloud

- **Selenium Grid** : Tests parallèles multi-navigateurs

2. Qu'est-ce que le TDD (Test-Driven Development) ?

Réponse :

Méthodologie de développement où **les tests sont écrits AVANT le code**.

Cycle TDD (Red-Green-Refactor) :

```
1. 🔴 RED
   Écrire un test qui ÉCHOUE
   (la fonctionnalité n'existe pas encore)
         ↓
2. ⚫ GREEN
   Écrire le code MINIMAL pour passer le test
   (faire passer le test au plus vite)
         ↓
3. 🔵 REFACTOR
   Améliorer le code sans casser les tests
   (nettoyage, optimisation, design)
         ↓
   Recommencer pour la prochaine fonctionnalité
```

Exemple concret :

Étape 1 : 🔴 RED (Test qui échoue)

```typescript
// calculator.test.ts
describe('Calculator', () => {
  it('should add two numbers', () => {
    const calc = new Calculator();
    expect(calc.add(2, 3)).toBe(5);  // ✗ FAIL: Calculator n'existe pas
  });
});
```

Étape 2 : ⚫ GREEN (Code minimal)

```typescript
// calculator.ts
class Calculator {
  add(a: number, b: number): number {
    return 5;  // ✅ PASS: Retourne 5 en dur (code minimal)
  }
}
```

Étape 3 : 🔵 REFACTOR (Amélioration)

```
// Ajouter un autre test qui échoue
it('should add 10 and 20', () => {
  const calc = new Calculator();
  expect(calc.add(10, 20)).toBe(30);  // ✗ FAIL: Retourne 5 au lieu de 30
});

// Améliorer le code pour généraliser
class Calculator {
  add(a: number, b: number): number {
    return a + b;  // ✅ Code correct et généralisé
  }
}
```

Avantages du TDD :

✅ **Couverture de tests élevée**

- 90-100% de code coverage
- Chaque ligne de code a un test associé

✅ **Design émergent**

- Le code est conçu pour être testable
- Interfaces claires et découplées
- Respect automatique des principes SOLID

✅ **Réduction des bugs**

- Détection immédiate des régressions
- Filet de sécurité pour le refactoring

✅ **Documentation vivante**

- Les tests documentent le comportement attendu
- Exemples d'utilisation concrets

✅ **Confiance pour refactorer**

- Modification du code sans crainte
- Tests vérifient que tout fonctionne toujours

Inconvénients :

× **Courbe d'apprentissage**

- Nécessite discipline et pratique
- Changement de mindset

× **Temps initial**

- Plus lent au début
- Gain de temps sur le long terme

× **Tests mal écrits**

- Tests trop couplés au code
- Tests fragiles qui cassent souvent

TDD vs Test-After :

Aspect	TDD	Test-After
Moment	Test AVANT code	Test APRÈS code
Coverage	90-100%	Variable (50-80%)
Design	Émergent, testable	Après coup
Bugs	Détection immédiate	Détection tardive
Refactoring	Facile et sûr	Risqué

Variantes du TDD :

BDD (Behavior-Driven Development)

- Extension du TDD
- Tests en langage naturel (Gherkin)
- Focus sur le comportement utilisateur

```
Given a calculator
When I add 2 and 3
Then the result should be 5
```

ATDD (Acceptance Test-Driven Development)

- Tests d'acceptation écrits par le métier
- Validation client continue

3. Comment tester une API REST ?

Réponse :

Stratégie de tests API complète :

1. Tests Unitaires (Logique métier) 🧩

```
// Tester la logique sans HTTP
describe('UserService', () => {
  it('should validate email format', () => {
    const service = new UserService();
    const result = service.isValidEmail('invalid-email');
    expect(result).toBe(false);
```

```
  });
});
```

2. Tests d'Intégration (Endpoints) 🔗

```
// Supertest (Node.js)
describe('POST /api/users', () => {
  it('should create user with valid data', async () => {
    const response = await request(app)
      .post('/api/users')
      .send({
        name: 'Alice',
        email: 'alice@test.com',
        password: 'SecurePass123!'
      })
      .expect(201);

    expect(response.body).toHaveProperty('id');
    expect(response.body.email).toBe('alice@test.com');
  });

  it('should return 400 for invalid email', async () => {
    const response = await request(app)
      .post('/api/users')
      .send({ email: 'invalid' })
      .expect(400);

    expect(response.body.error).toContain('Invalid email');
  });

  it('should return 409 for duplicate email', async () => {
    await createUser({ email: 'alice@test.com' });

    await request(app)
      .post('/api/users')
      .send({ email: 'alice@test.com' })
      .expect(409);
  });
});
```

3. Tests de Contrat (Contract Testing) 📜

Vérifier que l'API respecte son contrat (format, schéma).

```
// Pact (Consumer-Driven Contract Testing)
import { pactWith } from 'jest-pact';

pactWith({ consumer: 'Frontend', provider: 'UserAPI' }, (provider) => {
  it('should return user by ID', async () => {
```

```
    await provider.addInteraction({
      state: 'user 123 exists',
      uponReceiving: 'a request for user 123',
      withRequest: {
        method: 'GET',
        path: '/api/users/123'
      },
      willRespondWith: {
        status: 200,
        body: {
          id: 123,
          name: 'Alice',
          email: 'alice@test.com'
        }
      }
    });

    const response = await api.getUser(123);
    expect(response.id).toBe(123);
  });
});
```

4. Tests de Performance ⚡

```
// JMeter, K6, Artillery
// Vérifier temps de réponse sous charge

export const options = {
  thresholds: {
    http_req_duration: ['p(95)<500'],  // 95% < 500ms
    http_req_failed: ['rate<0.01'],    // < 1% d'erreurs
  },
};
```

5. Tests de Sécurité 🔒

```
describe('Security', () => {
  it('should require authentication', async () => {
    await request(app)
      .get('/api/users/me')
      .expect(401);
  });

  it('should prevent SQL injection', async () => {
    await request(app)
      .get('/api/users?id=1 OR 1=1--')
      .expect(400);
  });
```

```
    it('should sanitize XSS', async () => {
      const response = await request(app)
        .post('/api/comments')
        .send({ text: '<script>alert("XSS")</script>' })
        .expect(201);

      expect(response.body.text).not.toContain('<script>');
    });
  });
```

Checklist complète tests API :

Codes HTTP :

- ✅ 200 OK (succès GET)
- ✅ 201 Created (succès POST)
- ✅ 204 No Content (succès DELETE)
- ✅ 400 Bad Request (validation)
- ✅ 401 Unauthorized (non authentifié)
- ✅ 403 Forbidden (non autorisé)
- ✅ 404 Not Found (ressource absente)
- ✅ 409 Conflict (doublon)
- ✅ 422 Unprocessable Entity (erreur métier)
- ✅ 500 Internal Server Error (erreur serveur)

Headers :

- ✅ Content-Type: application/json
- ✅ Authorization: Bearer token
- ✅ Cache-Control
- ✅ CORS headers

Corps de réponse :

- ✅ Structure JSON conforme au schéma
- ✅ Données correctes
- ✅ Pas de données sensibles exposées

Outils de test API :

Outil	Usage
Postman	Tests manuels et automatisés, collections
Insomnia	Alternative à Postman
Supertest	Tests Node.js/Express
REST Assured	Tests Java/Spring Boot
pytest + requests	Tests Python

Outil	Usage
Pact	Contract testing
K6, JMeter	Performance

4. Qu'est-ce que l'intégration continue (CI) ?

Réponse :

CI (Continuous Integration) : Pratique d'intégration fréquente du code avec validation automatique.

Workflow CI typique :

```
Développeur
   ↓ (1. git push)
GitHub/GitLab
   ↓ (2. webhook)
CI Server (GitHub Actions, GitLab CI, Jenkins)
   ↓
┌──────────────────────────────┐
│  Pipeline CI                 │
│                              │
│  1. ✅ Checkout code         │
│  2. ✅ Install dependencies  │
│  3. ✅ Lint/Format check     │
│  4. ✅ Build application     │
│  5. ✅ Run unit tests        │
│  6. ✅ Run integration tests │
│  7. ✅ Security scan         │
│  8. ✅ Code coverage report  │
│  9. ✅ Build Docker image    │
│  10. ✅ Deploy to staging    │
└──────────────────────────────┘
   ↓
✅ Success → Merge autorisé
× Failure → Notification développeur
```

Exemple GitHub Actions :

```yaml
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
```

```yaml
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Lint code
        run: npm run lint

      - name: Run tests
        run: npm test
        env:
          DATABASE_URL: postgresql://postgres:postgres@localhost:5432/test

      - name: Coverage report
        run: npm run coverage

      - name: Upload coverage
        uses: codecov/codecov-action@v3

      - name: Build application
        run: npm run build

      - name: Security audit
        run: npm audit --production

  docker:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .
```

```
      - name: Push to registry
        run: docker push myapp:${{ github.sha }}
```

Principes de l'IC :

1. Intégrer fréquemment 🔄

- Commits quotidiens (minimum)
- Petits changements incrémentaux
- Branches courtes (< 2 jours)

2. Build automatique 🏗️

- Déclenchement à chaque commit
- Build reproductible
- Environnement isolé (conteneur)

3. Tests automatisés ✅

- Tests unitaires obligatoires
- Tests d'intégration
- Couverture de code > 80%

4. Feedback rapide ⚡

- Pipeline < 10 minutes
- Notifications immédiates (Slack, email)
- Indicateurs visuels (badges)

5. Tout le monde voit tout 👁️

- Dashboard public de l'équipe
- Historique des builds
- Tendances qualité

Avantages de la CI :

✅ Détection précoce des bugs

- Problèmes détectés en minutes/heures, pas en jours
- Coût de correction 10× moins élevé

✅ Intégration moins risquée

- Petits changements faciles à debugger
- Pas de "merge hell"

✅ Code toujours livrable

- Branche main toujours stable
- Déploiement possible à tout moment

✅ **Productivité**

- Moins de temps à corriger les bugs
- Focus sur les fonctionnalités

✅ **Qualité**

- Standards appliqués automatiquement
- Pas de code non testé en production

CI vs CD :

Aspect	CI	CD (Continuous Delivery)	CD (Continuous Deployment)
Objectif	Intégration validée	Livraison automatisée	Déploiement automatisé
Automatisation	Build + Tests	Build + Tests + Staging	Build + Tests + Production
Intervention manuelle	Aucune	Déploiement prod manuel	Aucune
Fréquence	À chaque commit	À chaque commit	À chaque commit

Outils CI populaires :

Outil	Points forts
GitHub Actions	Intégré GitHub, gratuit open-source
GitLab CI	Intégré GitLab, puissant
Jenkins	Open-source, très extensible
CircleCI	Rapide, cloud native
Travis CI	Simple, bon pour open-source
Azure DevOps	Intégration Microsoft

Métriques CI à suivre :

- **Build success rate** : > 90%
- **Build time** : < 10 minutes
- **Time to fix** : < 1 heure
- **Deploy frequency** : Multiple fois par jour
- **Lead time** : Commit → Production < 1 jour

Méthodologies de développement {#methodologies}

1. Comparez les méthodologies Agile et Waterfall

Réponse :

Waterfall (Cycle en V) 🌊 vs **Agile** ⚡

Waterfall - Approche séquentielle :

```
Exigences
   ↓ (validation)
Conception
   ↓ (validation)
Développement
   ↓ (validation)
Tests
   ↓ (validation)
Déploiement
   ↓
Maintenance
```

Agile - Approche itérative :

```
Sprint 1 (2-4 semaines)          Sprint 2          Sprint 3
┌──────────────────┐          ┌──────────┐      ┌──────────┐
│ Plan → Dev → Test│ → MVP →  │Dev → Test│ →    │Dev → Test│ → Produit
│      → Deploy    │          │ → Deploy │      │ → Deploy │    final
└──────────────────┘          └──────────┘      └──────────┘

      ↑     ↓                     ↑    ↓             ↑    ↓
   Feedback                    Feedback           Feedback
   utilisateur                 utilisateur        utilisateur
```

Comparaison détaillée :

Aspect	Waterfall 🌊	Agile ⚡
Approche	Séquentielle, phases distinctes	Itérative, incrémentale
Planning	Détaillé dès le début, fixe	Adaptatif par sprint
Exigences	Définies et gelées au début	Évolutives, raffinées continuellement
Feedback	Tardif (fin de projet)	Continu (chaque sprint)
Livraison	Une fois à la fin (6-24 mois)	Fréquente (2-4 semaines)
Changements	Difficiles et coûteux	Acceptés et intégrés
Documentation	Extensive et détaillée	Juste nécessaire, vivante
Tests	Phase dédiée à la fin	Continus (TDD, CI)
Risques	Élevés (effet tunnel)	Maîtrisés (itérations courtes)
Visibilité client	Faible jusqu'à la fin	Forte (démos régulières)

Aspect	Waterfall 🌊	Agile ⚡
Équipe	Spécialisée par phase	Pluridisciplinaire, auto-organisée
Contrat	Scope fixe, prix fixe	Time & Materials, scope flexible

Avantages Waterfall :

✅ **Structure claire**

- Phases bien définies
- Facile à comprendre
- Documentation complète

✅ **Prévisibilité**

- Planning détaillé
- Budget estimé
- Délais définis

✅ **Bon pour projets stables**

- Exigences bien connues
- Technologies maîtrisées
- Peu de changements attendus

Inconvénients Waterfall :

× **Rigidité**

- Changements très coûteux
- Retour en arrière difficile

× **Effet tunnel**

- Pas de visibilité pendant des mois
- Découverte tardive des problèmes

× **Risque d'obsolescence**

- Besoins évoluent pendant le projet
- Produit livré ne correspond plus au besoin

× **Feedback tardif**

- Client découvre le produit à la fin
- Corrections majeures possibles

Avantages Agile :

✅ **Flexibilité**

- Adaptation aux changements
- Priorisation continue

✅ **Feedback rapide**

- Validation régulière par le client
- Ajustements fréquents

✅ **Réduction des risques**

- Problèmes détectés tôt
- Livraisons fréquentes

✅ **Valeur continue**

- MVP rapidement disponible
- Fonctionnalités prioritaires d'abord

✅ **Qualité**

- Tests continus
- Refactoring régulier

Inconvénients Agile :

× **Moins prévisible**

- Scope évolutif
- Estimation complexe

× **Nécessite discipline**

- Pratiques rigoureuses
- Communication intensive

× **Client impliqué**

- Disponibilité requise
- Décisions fréquentes

Quand choisir Waterfall :

✅ **OUI si :**

- Exigences claires, stables et complètes
- Réglementation stricte (aéronautique, médical)
- Équipe distribuée avec peu de communication
- Technologie bien connue
- Exemple : Développement hardware/software embarqué

× **NON si :**

- Exigences floues ou évolutives
- Innovation ou nouveauté
- Besoin de feedback utilisateur

Quand choisir Agile :

✅ **OUI si :**

- Exigences évolutives
- Innovation, exploration
- Feedback utilisateur important
- Time-to-market critique
- Exemple : Applications web, mobile, SaaS

× **NON si :**

- Exigences réglementaires figées
- Contrat à prix fixe sans flexibilité
- Client non disponible

Approches hybrides :

Agile-Waterfall (Water-Scrum-Fall)

- Phases Waterfall (exigences, design)
- Développement Agile
- Déploiement Waterfall

SAFe (Scaled Agile Framework)

- Agile à l'échelle de l'entreprise
- Combine Scrum, Kanban, Lean

2. Expliquez la méthodologie Scrum

Réponse :

Scrum est un framework Agile pour gérer des projets complexes.

3 Rôles Scrum :

1. Product Owner (PO) 📋 Responsabilités :

- Définir la vision produit
- Gérer le Product Backlog (prioriser les fonctionnalités)
- Accepter ou rejeter le travail de l'équipe
- Représenter les utilisateurs et le métier

Activités :

- Rédiger les User Stories
- Définir les critères d'acceptation
- Participer au Sprint Planning
- Valider l'Increment lors de la Sprint Review

2. Scrum Master (SM) 👩‍🏫 Responsabilités :

- Faciliter les cérémonies Scrum
- Protéger l'équipe des perturbations externes

- Supprimer les obstacles (impediments)
- Coach Agile de l'équipe et de l'organisation

Activités :

- Animer les réunions Scrum
- Aider l'équipe à s'auto-organiser
- Promouvoir les pratiques Agile
- Améliorer les processus

3. Development Team 👫 Responsabilités :

- Développer le produit
- Auto-organisation
- Livrer un Increment potentiellement livrable

Caractéristiques :

- Pluridisciplinaire (dev, QA, UX, etc.)
- 3 à 9 personnes (idéalement 5-7)
- Pas de hiérarchie interne
- Responsabilité collective

Événements Scrum (Sprint de 1-4 semaines) :

0. Sprint 🏃

- Itération de durée fixe (souvent 2 semaines)
- Objectif : Livrer un Increment fonctionnel
- Pas de changement de Sprint Backlog en cours

1. Sprint Planning 📅 (4h pour sprint de 2 semaines)

Partie 1 - QUOI

- PO présente les User Stories prioritaires du Product Backlog
- Équipe sélectionne ce qu'elle peut livrer (Sprint Goal)

Partie 2 - COMMENT

- Équipe découpe les User Stories en tâches techniques
- Création du Sprint Backlog

Output : Sprint Backlog + Sprint Goal

2. Daily Scrum (Stand-up) 🗣️ (15 minutes max)

Format : Chaque membre répond :

1. Qu'ai-je fait hier ?
2. Que vais-je faire aujourd'hui ?
3. Quels obstacles rencontré-je ?

Règles :

- Même heure, même lieu
- Debout (pour rester concis)
- Synchronisation, pas reporting
- SM facilite mais n'interroge pas

3. Sprint Review (Démo) 🎯 (2h pour sprint de 2 semaines)

Objectif : Montrer l'Increment au PO et aux stakeholders

Activités :

- Démo des fonctionnalités terminées (Definition of Done respectée)
- PO accepte ou rejette
- Feedback des stakeholders
- Adaptation du Product Backlog

Output : Increment potentiellement livrable

4. Sprint Retrospective 🔄 (1h30 pour sprint de 2 semaines)

Objectif : Amélioration continue du processus

Format : Équipe discute :

- ✅ What went well ? (À continuer)
- ❌ What didn't go well ? (À améliorer)
- 💡 What can we improve ? (Actions)

Output : Plan d'actions concrètes pour le prochain sprint

Artefacts Scrum :

1. Product Backlog 📚

- Liste priorisée de toutes les fonctionnalités souhaitées
- Géré par le Product Owner
- Évolutif (raffinement continu)
- User Stories au format : "En tant que [rôle], je veux [action] afin de [bénéfice]"

Exemple :

```
1. 🔥 En tant qu'utilisateur, je veux me connecter avec email/mot de passe
afin d'accéder à mon compte (Story Points: 5)
2. 🔥 En tant qu'utilisateur, je veux réinitialiser mon mot de passe afin
de récupérer mon accès (SP: 3)
3. 🟡 En tant qu'admin, je veux voir le dashboard des ventes afin de suivre
le CA (SP: 8)
4. 🟢 En tant qu'utilisateur, je veux choisir un thème sombre afin de
confort visuel (SP: 2)
```

2. Sprint Backlog 📝

- Sous-ensemble du Product Backlog sélectionné pour le sprint
- Tâches techniques détaillées
- Géré par l'équipe

3. Increment 🎁

- Produit fonctionnel à la fin du sprint
- Respecte la Definition of Done
- Potentiellement livrable en production

Definition of Done (DoD) :

Checklist pour considérer une User Story terminée :

```
✅ Code développé
✅ Tests unitaires écrits et passés (> 80% coverage)
✅ Tests d'intégration passés
✅ Code review effectuée
✅ Documentation mise à jour
✅ Déployé en environnement de staging
✅ Accepté par le Product Owner
✅ Pas de bugs critiques
```

Estimation avec le Planning Poker :

```
Équipe estime la complexité en Story Points (suite de Fibonacci)
1, 2, 3, 5, 8, 13, 21, ...

Chaque développeur choisit une carte en secret
Révélation simultanée
Discussion si écarts importants
Consensus
```

Burndown Chart :

```
Story Points
    ↑
 50 | *
    |   *
 40 |     *
    |       *
 30 |         *  ← Idéal
    |           * *
 20 |             * *
    |               * *
 10 |                 * *
```

```
    |                    *
  0 |──────────────────────→ Jours
    0  2  4  6  8  10 12 14

* = Travail restant réel
```

Vélocité :

Mesure de la productivité de l'équipe (Story Points complétés par sprint)

```
Sprint 1: 18 SP
Sprint 2: 22 SP
Sprint 3: 20 SP
→ Vélocité moyenne: 20 SP

Prévision: Avec 100 SP restants → 5 sprints nécessaires
```

Avantages de Scrum :

✅ **Transparence**

- Tout le monde voit l'avancement
- Problèmes visibles rapidement

✅ **Inspection**

- Revue régulière du produit et du processus
- Adaptation continue

✅ **Adaptation**

- Changements intégrés à chaque sprint
- Priorisation flexible

✅ **Livraison de valeur**

- Fonctionnalités utilisables toutes les 2-4 semaines
- Feedback rapide

3. Qu'est-ce qu'un MVP (Minimum Viable Product) ?

Réponse :

MVP = Version minimale d'un produit avec juste assez de fonctionnalités pour :

1. Satisfaire les premiers utilisateurs (early adopters)
2. Valider les hypothèses métier
3. Apprendre rapidement

Concept :

```
× NE PAS FAIRE (Big Bang)
Développement 12 mois → Produit complet → Lancement

✅ FAIRE (MVP Itératif)
MVP v1 (2 mois) → Apprendre → MVP v2 (1 mois) → Apprendre → Produit
  ↓ Feedback       ↓ Pivot          ↓ Feedback         ↓
Utilisateurs     Ajustement      Utilisateurs    Croissance
```

Exemple concret - Spotify :

Pas MVP :

- × Application complète avec 50M de titres, playlists, radio, podcasts, lyrics, social...
- 💰 Coût : 5M€, 24 mois
- 🎲 Risque : Personne ne veut le produit

MVP v1 (3 mois) :

- ✅ Écouter 1000 titres en streaming
- ✅ Recherche basique
- ✅ Créer une playlist

Hypothèses à valider :

- Les gens veulent-ils streamer au lieu de télécharger ?
- Sont-ils prêts à payer un abonnement ?

Métriques :

- 1000 utilisateurs inscrits en 1 mois
- 60% utilisent l'app au moins 1× par semaine
- ✅ Hypothèse validée → Continuer

MVP v2 (2 mois) :

- ✅ 10 000 titres
- ✅ Partage de playlists
- ✅ Mode offline

Caractéristiques d'un bon MVP :

1. Minimum 🎯

- Fonctionnalités essentielles uniquement
- Pas de "nice to have"
- Développement rapide (6-12 semaines max)

2. Viable ✅

- Produit fonctionnel, pas prototype
- Utilisable par de vrais utilisateurs

- Valeur claire livrée

3. Product 🎁

- Résout un problème réel
- Expérience utilisateur acceptable (pas parfaite)
- Peut générer des revenus (freemium, early access, etc.)

Erreurs courantes :

× **MVP trop gros**

- Trop de fonctionnalités
- Trop de temps de développement
- Risque d'apprendre trop tard

× **MVP trop petit**

- Pas assez de valeur
- Utilisateurs frustrés
- Pas de signal valide

× **Prototype ≠ MVP**

- Prototype : Démonstration, non fonctionnel
- MVP : Produit utilisable en production

Approche Build-Measure-Learn (Lean Startup) :

```
┌──────────┐
│  IDEAS   │
└──────────┘
     │
     ↓
┌──────────┐
│   BUILD  │ ← Créer MVP
│   MVP    │
└──────────┘
     │
     ↓
┌──────────┐
│  MEASURE │ ← Métriques
│ Analytics│
└──────────┘
     │
     ↓
┌──────────┐
│   LEARN  │ ← Insights
│ Insights │
└──────────┘
     │
     ↓
Persevere ou Pivot ?
     ↓
Recommencer le cycle
```

Métriques MVP (Pirate Metrics - AARRR) :

- **Acquisition** : Comment les gens découvrent le produit ?
- **Activation** : Première expérience positive ?
- **Retention** : Reviennent-ils ?
- **Referral** : Recommandent-ils ?
- **Revenue** : Paient-ils ?

Exemples MVP célèbres :

Dropbox :

- MVP = Vidéo de démo (3 minutes)
- Pas de produit fonctionnel
- 75 000 inscriptions en une nuit
- Validation : Les gens veulent ce produit

Airbnb :

- MVP = Photos de leur propre appartement
- Site web basique
- Validation manuelle
- 3 locations → Concept validé

Amazon :

- MVP = Librairie en ligne uniquement
- Garage de Jeff Bezos
- Commandes manuelles
- Puis expansion progressive

Différence MVP vs Prototype vs PoC :

	Proof of Concept (PoC)	**Prototype**	**MVP**
Objectif	Faisabilité technique	Design/UX	Validation marché
Public	Équipe technique	Stakeholders internes	Utilisateurs réels
Qualité	Code jetable	Non production	Production ready
Durée	1-2 semaines	2-4 semaines	6-12 semaines
Exemple	"Peut-on intégrer l'API X ?"	Maquette cliquable Figma	App déployée avec signup

---

[La suite du vademecum continue avec Performance, APIs, DevOps, Technologies, Git, Design Patterns, WebSockets, PWA et OAuth dans le prochain message]